

Verifier-Augmented vs. LLM-Only NetOps Agents: A Paired Evaluation on an Action-Aware Staging Benchmark

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-backed paired evaluation comparing a verifier-augmented NetOps pipeline to an LLM-only pipeline on a compact action-aware staging benchmark (ActionAwareStagingBench-v1). The preregistered design used gpt-5-mini as the generator, deterministic_netops_validator_v5 as the runtime guard and post-hoc scorer, N=400 paired tasks (800 episodes), and shared_initial_generation semantics (one shared LLM candidate per task delivered to both conditions). Confirmatory outcomes (archived artifacts) show complete_pairs = 400; baseline_success_count = 400/400; guarded_success_count = 400/400; paired_contingency n_11 = 400, n_10 = n_01 = n_00 = 0; observed paired difference = 0.0; preregistered permutation p = 1.0; exact McNemar p = 1.0; paired bootstrap 95% CI = [0.0, 0.0] (bootstrap_seed = 1729; 10,000 resamples). We present the preregistered methodology, the exact artifact-grounded results, a diagnostic analysis showing that shared_initial_generation together with reuse of the same deterministic validator produced a deterministic ceiling that rendered the primary estimand uninformative in this run, and concrete artifact-grounded recommendations to harden future benchmarks so they expose generator×guard interactions.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into network configuration artifacts and staged change workflows (canaries, progressive rollout, rollback). Prior work shows LLMs can produce syntactically plausible configuration templates in controlled settings, and separate lines of research demonstrate that verifier-guided generate-validate-repair loops and deterministic network verifiers can reduce syntactic and semantic errors in infrastructure-as-code (IaC) and related domains. The operational safety question for agentic NetOps pipelines is end-to-end: how do generator variability, runtime verification, staged rollout semantics, repair, and rollback interact to affect safe-deploy outcomes?

Research question. Under the benchmark semantics used here, does integrating a deterministic runtime validator with an LLM generator increase the paired safe-deploy rate relative to an LLM-only pipeline? The preregistered primary estimand was the paired difference in safe-deploy rate (guarded - baseline) measured on N=400 preregistered paired tasks.

Contributions and summary. (1) We executed a fully preregistered paired evaluation (cnsmauto-3d2k-prereg-v1) using gpt-5-mini and deterministic_netops_validator_v5 and archived every runtime trace, validator_trace, and analysis

artifact. (2) The exact artifact-grounded confirmatory result is a deterministic ceiling: every shared generator output was accepted by the deterministic validator yielding baseline and guarded success rates of 1.0 (paired difference = 0.0; permutation p = 1.0). (3) We diagnose why the preregistered design produced this ceiling (shared_initial_generation + reuse of the same deterministic validator for runtime guard and scoring), and provide artifact-grounded benchmark-hardening recommendations (independent generation per condition, distinct scorer, difficulty-stratified sampling, controlled adversarial seeding, and instrumented mitigation metrics). These recommendations are implementable using the archived task manifest and validator traces.

II. RELATED WORK

This study combines three intersecting literatures. Intent→configuration generation: recent works explore natural-language intent translation into network or infrastructure configuration templates and demonstrate promising prototype results on curated datasets and testbeds (e.g., Nguyen et al., "Intent-Based Network Configuration Using Large Language Models", 2024; Li et al., "PreConfig", 2024; Zhang et al., "GraphSynth", 2025). Verifier-guided workflows and verifier-guided fine-tuning report correctness gains for IaC and provisioning tasks (e.g., Jana et al., "TerraFormer", 2026). Deterministic network verification and reachability tools are effective in finding configuration errors (e.g., Liu et al., "Scalable and Interpretable Overlay Network Checking via Ensemble Verification", 2025; Koc, "Reachability Modeling for Kubernetes Network Policies", 2026). Agentic and tool-augmented LLM architectures (ReAct, RAG, multi-agent orchestrations) expand AIOps/RCA functionality but add failure modes—hallucination, misapplied tools, miscalibrated confidence—that motivate runtime guardrails and staged deployments (e.g., Pei et al., "Flow-of-Action", 2025; Taparia et al., "Learning to Configure Agentic AI Systems", 2026). Surveys identify an evaluation gap: action-aware benchmarks that combine generation, runtime blocking, staged rollout, repair, rollback, and provenance are scarce (see Rahman & Swapnil, 2026; Hong et al., 2025). Our contribution is an explicit, preregistered, artifact-backed evaluation that operationalizes a compact action-aware staging benchmark and exposes a practical design pitfall.

III. METHODOLOGY

Preregistration and estimands. The study preregistration (cnsmauto-3d2k-prereg-v1) specified a paired binary design with exactly $N=400$ paired tasks, primary estimand `paired_success_rate_difference_guarded_minus_baseline`, and a permutation-based paired difference test as the confirmatory procedure. An exact McNemar two-sided test was also computed for corroboration. Secondary preregistered outcomes included bootstrap CIs for median time-to-rollback and counts of repair attempts.

Adapter semantics and critical design choice. Execution used the `hosted_netops_gvr_v1` adapter with declared generator `gpt-5-mini` and `deterministic_netops_validator_v5` as both runtime guard and post-hoc scorer. Critically, preregistration set `shared_initial_generation = true` (`independent_condition_generation = false`): for each task we obtained one deterministic LLM candidate and delivered that same candidate to both baseline and guarded episodes. This mapping intentionally isolates the validator’s acceptance decision on identical generator outputs and reduces model-call budget (`model_calls_used = 400` for 800 episodes). The preregistered repair policy permitted up to one repair attempt in the guarded episode; `repair_applied` flags and repair traces were recorded.

Task generation, deterministic replay, and scoring. Tasks were produced deterministically by `deterministic_netops_task_generator_v5` and carry structured difficulty metadata (`changed_interface_count`, `dependency_rule_count`, `pattern`, `required_command_count`, `level` $\in$ {medium, hard, very_hard}). The preregistration required three-replay deterministic consistency checks per agent+simulator pair; failing pairs or simulator errors would be excluded. Scoring used `deterministic_netops_validator_v5` applying full intent-constraint validation; `validator_trace` entries record `validation_before/after`, `final_configuration`, and validity decision.

IV. RESULTS

Execution accounting and reconciliation. The run started 2026-08-29T18:23:37Z and completed 2026-08-29T19:12:38Z. Planned episodes = 800; completed episodes = 800; failed episodes = 0; `model_calls_used` = 400 (shared initial generation); `complete_pairs` = 400. Deterministic three-replay checks passed for all pairs; no exclusions were recorded.

Primary confirmatory outcomes (artifact-grounded). The archived contingency counts are `baseline_success_count` = 400 (`baseline_success_rate` = 1.0) and `guarded_success_count` = 400 (`guarded_success_rate` = 1.0). The paired contingency table is $n_{11} = 400$ with $n_{10} = n_{01} = n_{00} = 0$, yielding observed paired difference (`guarded` - `baseline`) = 0.0. The preregistered permutation test returned $p = 1.0$; the exact McNemar two-sided test returned $p = 1.0$. The paired bootstrap 95% CI = [0.0, 0.0] (`bootstrap_seed` = 1729; 10,000 resamples). Cohen’s $h = 0.0$. These numbers are reproduced directly from the archived analysis artifacts (the archived

paired contingency table, the archived condition summary, the archived analysis artifact).

Repair and rollback observations. Across the 800 episodes `repair_applied` = false and no rollbacks were invoked; `validator_trace` entries show `validation_after.final_state` matched `expected_final_state` for accepted configurations. Thus, when the shared candidate satisfied `deterministic_netops_validator_v5` both episodes necessarily recorded safe-deploy outcomes.

Aggregated difficulty summary (exact, artifact-extracted). We aggregated task difficulty labels from the archived task manifest (the archived task manifest; artifact hash: `ca02bdfa2c5f526c8588424a11ce5bcf9915a41c16bf2c96b9e53f4726f7ea37`). The manifest labels for the $N=400$ tasks yield the distribution shown in Table 1 below. This aggregation is reported verbatim from the archived manifest.

Table 1 (Difficulty distribution extracted from the archived task manifest): - medium: 80 tasks (20.0%) - hard: 160 tasks (40.0%) - very_hard: 160 tasks (40.0%)

(Task manifest and per-task difficulty fields are archived for auditors at the archived task manifest.)

Statistical interpretation. The observed deterministic ceiling collapses inferential variance: permutation and McNemar correctly report $p = 1.0$, but those statistics do not assess generator $\times$ guard closed-loop dynamics because the design prevented the guard from having an opportunity to intervene. The preregistered power calculation assumed a non-ceiling baseline failure rate and within-pair variability sufficient to detect an absolute difference ≈ 0.10 with $N=400$; those assumptions are invalidated here and so the original power rationale does not apply.

V. DISCUSSION

Mechanism of the ceiling effect. Two preregistered design choices produced the deterministic ceiling. First, `shared_initial_generation` removed generator variance by delivering identical LLM outputs to baseline and guarded episodes. Second, the runtime guard and post-hoc scorer were the same deterministic implementation (`deterministic_netops_validator_v5`). When the deterministic validator accepts the shared candidate, both conditions record identical safe-deploy outcomes and the paired difference must be zero. This is a construct-validity outcome of the frozen design, not a general null statement about verifier utility across generator/validator pairings.

Implications for experimental design and evaluation. Evaluations intended to measure runtime guard utility must preserve opportunities for the guard to accept, block, or request repair. From the archived artifacts we propose concrete, implementable hardening steps: (1) independent per-condition generation (`independent_condition_generation = true`), (2) use a distinct post-hoc scorer or held-out adjudicator, (3) controlled adversarial seeding to guarantee nontrivial baseline failure rates, (4) difficulty-stratified sampling to ensure enough `very_hard` tasks, and (5) instrumented mitigation metrics (`repair_attempts`, `repairs_applied`, `time-to-rollback`). All of these

recommendations are practical because the archived task manifest supports deterministic seeding and difficulty labels and the `validator_trace` already records repair/rollback fields.

Threats to validity and limitations. Internal validity: shared generation and reuse of the same validator conflated runtime acceptance and post-hoc scoring. Construct validity: the deterministic validator approximates production checks but does not capture telemetry noise or asynchronous state. External validity: single generator (gpt-5-mini) and single validator limit generalization. Conclusion validity: absence of within-pair variation invalidates the preregistered power rationale. Adversarial robustness: adversarial or distribution-shifted prompts were not exercised and remain an open empirical question. See Limitations section for the full list.

VI. LIMITATIONS

- Benchmark hardness and ceiling effect: under preregistered `shared_initial_generation` semantics and the chosen generator+validator, every shared candidate satisfied `deterministic_netops_validator_v5` so the guard had no opportunity to alter outcomes; this makes the primary estimand effectively untestable in this run and invalidates the original power rationale.
- Model and scope: the experiment used a single generator (gpt-5-mini) and a single deterministic validator (`deterministic_netops_validator_v5`); results do not generalize across models, validators, or production heterogeneity.
- Construct validity: the same `deterministic_netops_validator_v5` implementation served as both runtime guard and the post-hoc scorer; this conflation produces concordant acceptance/scoring and limits inference about independent closed-loop verifier utility.
- Shared candidate semantics: preregistered `shared_initial_generation` (one LLM generation per task shared by both conditions) intentionally reduces generation variance and therefore limits inference to the validator’s behavior on identical candidates rather than full generator+guard closed-loop interactions.
- Synthetic environment and external validity: the simulator and validator do not fully model production telemetry noise, operator workflows, or adversarial actor behaviours; external validity to live networks is limited.
- Adversarial robustness untested: adversarial or distribution-shifted prompts and generator faults that could trigger guard interventions were not exercised in this run and remain open for future work.
- Aggregated difficulty reporting: the full per-task difficulty labels for all $N=400$ are archived in the archived task manifest and can be aggregated by auditors; Table 1 reports the artifact-extracted aggregated counts and percentages.

VII. CONCLUSION

We executed a fully preregistered paired evaluation comparing a verifier-augmented NetOps pipeline to an LLM-only

pipeline on an action-aware staging benchmark. Under the preregistered `shared_initial_generation` semantics and the chosen generator+validator pairing, every shared candidate satisfied `deterministic_netops_validator_v5` and both conditions produced 100% safe-deploy across $N=400$ pairs (paired difference = 0.0; permutation $p = 1.0$). This deterministic ceiling resulted from frozen design and implementation homogeneity and renders the primary estimand uninformative for generator $\times$ guard closed-loop interactions in this run. We archive the preregistration and full the archived execution artifact artifacts to enable reproducibility and to support hardened follow-on experiments (independent generation, adversarial seeding, heterogeneous validators) that exercise blocking, repair, and rollback behaviors. The artifact-grounded hardening recommendations supplied here offer concrete steps researchers can preregister to preserve statistical signal when evaluating runtime verifier utility in NetOps settings.

Reproducibility and artifacts. All execution and analysis artifacts supporting the numbers and traces reported here are archived in the bundled artifact set (the archived execution artifacts and the archived analysis artifacts). Key artifacts include the execution manifest, raw results, model configuration, deterministic validator traces, the task manifest with per-task difficulty metadata, and the analysis results (paired contingency and bootstrap outputs). Auditors can reproduce the reported summaries using the archived manifests and analysis code as recorded in the artifact bundle.

Disclosure Statement (mandatory): Declared generator: gpt-5-mini. Orchestration: `hosted_netops_gvr_v1` adapter family with `shared_initial_generation` semantics (`independent_condition_generation = false`) as preregistered. Runtime guard and post-hoc scorer: `deterministic_netops_validator_v5`. Task generator: `deterministic_netops_task_generator_v5`. Preregistration: `cnsmauto-3d2k-prereg-v1` (preregistration SHA-256 recorded in the execution manifest). The immutable initial master prompt is archived at the archived provenance artifact with SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba.

Topic constraints were selected pre-lock by the Research Director and limited the study to Generative AI and LLMs for NetOps focused on reliability/safety of intent $\rightarrow$ configuration generation. Permitted tools and capabilities: hosted model API calls, `deterministic_netops_validator_v5` for runtime guarding and post-hoc scoring, `deterministic_netops_task_generator_v5` for task generation, `paired_binary_analysis_v1` for confirmatory analysis. The guarded condition routed the shared LLM candidate to `deterministic_netops_validator_v5` which deterministically accepted or blocked candidates per the preregistered adapter policy; the adapter allowed up to one repair call per task but no repairs were applied (`repair_applied = false` across episodes). All runtime calls, raw outputs, validator traces, deterministic three-replay checks, exclusion logs, and analysis artifacts are archived in the artifact bundle (the archived execution artifacts and the archived analysis artifacts). Autonomous literature review, experiment selection, preregistration, execution, analysis, peer

review, revision, and manuscript drafting occurred post-lock under the master prompt autonomy constraints; no human manually edited the technical manuscript content after the autonomy lock.

Limitations: [1] Benchmark hardness and ceiling effect: under preregistered `shared_initial_generation` semantics and the chosen generator+validator, every shared candidate satisfied `deterministic_netops_validator_v5` so the guard had no opportunity to alter outcomes; this makes the primary estimand effectively untestable in this run and invalidates the original power rationale. [2] Model and scope: the experiment used a single generator (gpt-5-mini) and a single deterministic validator (`deterministic_netops_validator_v5`); results do not generalize across models, validators, or production heterogeneity. [3] Construct validity: the same `deterministic_netops_validator_v5` implementation served as both runtime guard and post-hoc scorer; this conflation produces concordant acceptance/scoring and limits inference about independent closed-loop verifier utility. [4] Shared candidate semantics: preregistered `shared_initial_generation` reduces generation variance and therefore limits inference to the validator's behavior on identical candidates rather than full generator+guard closed-loop interactions. [5] Synthetic environment and external validity: the simulator and validator do not fully model production telemetry noise, operator workflows, or adversarial actor behaviours. [6] Adversarial robustness untested: adversarial or distribution-shifted prompts were not exercised and remain open for follow-on preregistered work. [7] Aggregated difficulty reporting: the full per-task difficulty labels for all N=400 are archived in the archived task manifest and the aggregated counts included in Table 1 are extracted verbatim from that manifest (artifact hash: `ca02bdfa2c5f526c8588424a11ce5bcf9915a41c16bf2c96b9e53f4726f7ea37`). Auditors may recompute or reaggregate these labels using the archived manifest.

References: The manuscript cites and relies on the following verified sources (selected nearest works referenced in Related Work):

- [1] N. Van Tu, S. Nam, and J. W.-K. Hong, "Intent-Based Network Configuration Using Large Language Models," *Network and Emerging Markets*, 2024. 10.1002/nem.2313.
- [2] F. Li, H. Lang, J. Zhang, J. Shen, and X. Wang, "PreConfig: A Pretrained Model for Automating Network Configuration," *arXiv preprint*, 2024. URL: <http://arxiv.org/abs/2403.09369>.
- [3] P. Jana et al., "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback," *Proceedings of the 2026 Conference on Systems*, 2026. 10.1145/3786583.3786898.
- [4] C. Pei et al., "Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis," *Proceedings of the 2025 ACM Conference*, 2025. 10.1145/3701716.3715225.
- [5] X. Liu et al., "Scalable and Interpretable Overlay Network Checking via Ensemble Verification," 2025. 10.1145/3768974.

- [6] R. Rahman and R. R. Swapnil, "Security Evaluation Gaps in Retrieval-augmented and Agentic LLM Systems: A Focused Review," *SSRN*, 2026. 10.2139/ssrn.7023538.

(Complete verified bibliography and metadata are archived in the manuscript evidence bundle.)

DISCLOSURE STATEMENT

Disclosure Statement (mandatory): Declared generator: gpt-5-mini. Orchestration: hosted_netops_gvr_v1 adapter family with `shared_initial_generation` semantics (`independent_condition_generation = false`) as preregistered. Runtime guard and post-hoc scorer: `deterministic_netops_validator_v5`. Task generator: `deterministic_netops_task_generator_v5`. Preregistration: `cnsmauto-3d2k-prereg-v1` (preregistration SHA-256 recorded in the execution manifest). The immutable initial master prompt is archived at `provenance/master_prompt.txt` with SHA-256 = `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba`. Topic constraints were selected pre-lock by the Research Director and limited the study to Generative AI and LLMs for NetOps focused on reliability/safety of intent→configuration generation. Permitted tools and capabilities: hosted model API calls, `deterministic_netops_validator_v5` for runtime guarding and post-hoc scoring, `deterministic_netops_task_generator_v5` for task generation, `paired_binary_analysis_v1` for confirmatory analysis. The guarded condition routed the shared LLM candidate to `deterministic_netops_validator_v5` which deterministically accepted or blocked candidates per the preregistered adapter policy; the adapter allowed up to one repair call per task but no repairs were applied (`repair_applied = false` across episodes). All runtime calls, raw outputs, validator traces, deterministic three-replay checks, exclusion logs, and analysis artifacts are archived in the artifact bundle (`execution/*` and `analysis/*`). Autonomous literature review, experiment selection, preregistration, execution, analysis, peer review, revision, and manuscript drafting occurred post-lock under the master prompt autonomy constraints; no human manually edited the technical manuscript content after the autonomy lock.

REFERENCES

- [1] Nguyen Van Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369
- [3] Prithwish Jana, Sam Davidson, Bhavana Bhasker, Andrey Kan, Anoop Deoras, Laurent Callot, "TerraFormer: Automated Infrastructure-as-Code with LLMs Fine-Tuned via Policy-Guided Verifier Feedback", 2026, 10.1145/3786583.3786898
- [4] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, Gaogang Xie, Dan Pei, "Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis", 2025, 10.1145/3701716.3715225
- [5] Xinzhe Liu, Yahui Li, Han Zhang, Xia Yin, Xingang Shi, Zhiliang Wang, Gang Ren, Jilong Wang, Jiangyuan Yao, "Scalable and Interpretable Overlay Network Checking via Ensemble Verification", 2025, 10.1145/3768974

- [6] Toghrul Abbasli, Kentaroh Toyoda, Yuan Wang, Li Chen, "Claim-Level Confidence Calibration for Reliable Decision Making with Large Language Models", 2026, 10.48550/arxiv.2608.22483
- [7] Rafi Rahman, Rimu Rahman Swapnil, "Security Evaluation Gaps in Retrieval-augmented and Agentic LLM Systems: A Focused Review", 2026, 10.2139/ssrn.7023538
- [8] doi:10.1145/3786583.3786898